Algorithm 1: BeamSearch

Input: $G = (\mathcal{X}, E)$ a graph over set of points $\mathcal{X}$ fitted with dissimilarity measure $\|\cdot,\cdot\|$, $s \in \mathcal{X}$ a source point, q a target point, $L \in \mathbb{N}$ the beam width parameter

Output: $B \subseteq \mathcal{X}$ the L closest points to q encountered during the search

$B \leftarrow \{s\}$;
$V \leftarrow \emptyset$;
while $B \setminus V \neq \emptyset$ **do**
  $p \leftarrow \text{argmin}_{u \in B \setminus V} \|u, q\|$;
  $B \leftarrow B \cup \texttt{Neighbors}(curr)$;
  $V \leftarrow V \cup \{curr\}$;
  if $|B| > L$ **then**
    $B \leftarrow$ closest L points to q in B;
return B;

Pruning algorithms are typically derived from the construction method of the Relative Neighborhood Graph (RNG), which selects edges based on the proximity of candidates to one another [3]. Insofar as pruning requires doing pairwise comparisons between candidate neighbors, these pruning rules are inherently quadratic in the number of candidates. Consequently, one cannot simply prune the entire universe of n points for every vertex, as the resulting $O(n^3)$ construction time would be prohibitive for large datasets.

Many methods circumvent this issue by way of an *incremental construction paradigm*: for each point being inserted, the algorithm performs a search on the current (partial) index to identify a local collection of candidate neighbors. These candidates are then refined using a pruning kernel, such as the RobustPrune algorithm used in Vamana [24] (see Algorithm 2), the RNG-based rules in HNSW and NSG [16, 29], or the direction-based pruning strategy in SSG [15]. While effective at identifying relevant edges, incremental construction suffers from the *search bottleneck*, where the majority of build time is spent on latency-bound random memory accesses that fail to exploit modern hardware parallelism.

Recent attempts to achieve faster index construction try to reduce the cost of these searches. MIRAGE [41] aims to improve the quality of the base graph in order to make the searches more efficient. LSH-APG [47] uses LSH to avoid some distance computations in the search, as well as to pick a closer starting point. FastKCNA [43] introduces a 'refinement-before-search' framework. Rather than searching a partial graph to find candidates for pruning, it first applies a relaxed RNG constraint (α-pruning) to an approximate KNN graph, yielding faster searches for identifying candidate neighbors. However, these methods do not eliminate searches entirely, and thus still suffer from the associated inefficiencies.

An alternative to the search-based approach is to use some kind of overlapping partitioning, as seen in methods like HCNNG [32]. *Partitioning approaches* identify potential neighbors without searching the graph which allows for more cache-friendly algorithms via batched distance computations. However, the curse of dimensionality forces clustering methods to generate a significantly larger universe of candidate points per vertex than search-based methods. Storing all these candidates for a final "offline" prune would require a massive memory footprint, and the resulting candidate lists would be prohibitively large to prune using existing $O(n^2)$ pruning methods. Moreover, existing partitioning approaches do not consistently produce state of the art ANNS indexes. This is because, absent a viable pruning strategy, HCNNG combines edges produced in each cluster via union, which yields dense adjacency lists with directional redundancy.

The ideal solution for a partitioning algorithm would be to prune candidates as they are discovered in each partition. However, existing pruning kernels do not lend themselves to online, batched execution. Firstly, each batch of candidates would require a quadratic number of distance computations over the union of the batch and existing partial adjacency list, which is computationally prohibitive. Moreover, if a candidate list is divided into arbitrary batches (e.g., partition by partition), traditional pruning rules lose global coherence. For example, a point z might be pruned in an early batch because of an "intermediary" point y. If y is itself later pruned during the processing of another partition, the algorithm no longer has a justification for the pruning of z, potentially leading to a fragmented graph structure. Thus, existing pruning methods lack the history-independence required to ensure that the final graph structure remains consistent regardless of the order that candidates are processed. These challenges motivate our development of HashPrune, which maintains a deterministic and sparse graph across arbitrary collections of candidates while simultaneously guaranteeing a bounded memory footprint and fast updates.

Algorithm 2: RobustPrune

Input: $\mathcal{N}$ a collection of candidate points fitted with dissimilarity measure $\|\cdot,\cdot\|$, points $x \in \mathcal{X}$, $\alpha \in \mathbb{R}$, $R \in \mathbb{N}$

Output: $E \subseteq \mathcal{X} \times \mathcal{X}$

while $\mathcal{N} \neq \emptyset$ *and* $|E| < R$ **do**
  $y \leftarrow \text{argmin}_{y \in \mathcal{X}} \|x, y\|$;
  E.Add$((x, y))$;
  $\mathcal{N}$.Remove(y);
  for $z \in \mathcal{N}$ **do**
    if $\alpha \cdot \|y, z\| < \|x, z\|$ **then**
      $\mathcal{N}$.Remove(z)
return E;

3 ONLINE PRUNING VIA HASHPRUNE

In this section, we devise a history-independent online pruning technique which we call ***HashPrune*** which is integral to the partitioning approach used in PiPNN.

Motivation. As mentioned in Section 2.1, pruning algorithms ensure that each vertex connects to neighbors in many different directions so that greedy search can quickly route from any one point to any other. HashPrune accomplishes this objective via locality-sensitive hashing (LSH) [12] to prune candidate edges which are likely to be directionally similar to other candidates. The use of LSH also allows for the use of low dimensional *sketches* during pruning, which eliminates the need to fetch high dimensional vectors. We give pseudocode for the HashPrune algorithm in Algorithm 3, and formally describe the components of HashPrune next.